Java: Airport Baggage Handling System

Hard

Overview

Implement an airport baggage sorting system that efficiently categorizes items by destination using parallel processing. Each baggage item, identified by its name and destination, should be grouped accordingly, leveraging the Fork-Join Framework for parallel sorting. Goal Note: Example Explanation: Output: Explanation: The bags are categorized into two destinations: "Berlin" and "Paris". The following sequence of events unfolds: Since BagC arrives before BagB, the destination "Berlin" will contain bags in the following order [BagC, BagB]. Only one baggage item has the destination "Paris"; thus, it will contain [BagA]. Classes and Methods: class BaggageItem : Each object of this class contains: class BaggageSorter extends RecursiveTask<Map<String, List<BaggageItem>>>: It handles parallel sorting of baggage items using the Fork-Join Framework, extending RecursiveTask for recursive task decomposition. Its attributes are: Map<String, List<BaggageItem>> sortBaggage(int numberOfBags, List<BaggageItem> baggageItems): The method manages parallel sorting of baggage items. Function Description Complete the function compute() in the editor: It sorts the baggage items based on their destinations either using sequential sorting or in parallel using the Fork-Join Framework. Function Parameters: No parameters, but the class has access to baggageItems, start, and end to define the scope of work for each subtask. Returns Map<String, List<BaggageItem>>: where the key is a destination, e.g., "Paris", and the value is a list of baggage items assigned to that destination. Constraints The first line contains the numberOfBags. The second line contains the numberOfDestinations. The next numberOfDestinations lines include the destination names. The next 2*numberOfBags lines contain baggage items. For each baggage item, two lines follow: The first line contains the name. The second line contains the destination. Sample Input 0 Sample Output 0 Explanation Sample Input 1 Sample Output 1 Explanation INPUT FORMAT FOR CUSTOM TESTING SAMPLE CASE 0 SAMPLE CASE 1 Sort a list of baggage items by their assigned destinations using either sequential sorting (sorting via ascending order of their arrival time) or a divide-and-conquer approach based on the constraints outlined in the compute() function description.Display each destination, sorted alphabetically in ascending order, along with its assigned baggage names sorted as per the given defined rule.It is guaranteed that there are no duplicate baggage or destination names.Each bag's destination is included in the given list of destinations. numberOfBags: 3 numberOfDestinations: 2 Destinations: ["Paris", "Berlin"] Bag Names: ["BagA", "BagB", "BagC"] Destinations for each Bag: ["Paris", "Berlin", "Berlin"] Since there are fewer than 10 bags, sorting is sequential. BagA arrives 1st with its destination as "Paris".BagC arrives 2nd with its destination as "Berlin".BagB arrives 3rd with its destination as "Berlin". name: The bag's name, e.g., "BagA". destination: The bag's destination, e.g., "Paris". baggageItems: A list of all baggage items to sort. start: The start index of the list for the current task. end: The end index of the list for the current task. It creates a ForkJoinPool. It invokes the BaggageSorter task to perform parallel sorting. It returns a map with destination names as keys and sorted lists of corresponding baggage items as values. If the task is small (10 or fewer baggage items): Sort sequentially i.e., sorting as per the baggage arrival time. If there are more than 10 baggage items, split the task into left and right subtasks, sort both in parallel, merge the left into the right, i.e., right = right + left, and return the right subtask as the final sorted result. $1 \leq$ numberOfBags ≤ 30 $1 \leq$ numberOfDestinations ≤ 20 Each bag's name will be a non-empty string that may contain a single whitespace with 1 to 20 alphanumeric characters. The baggage is categorized into 2 destinations: Miami and Dallas. Since the number of items is less than 10, sorting is performed sequentially. There are 3 baggage items: BagX, BagY, and BagZ. There are 2 destinations: Tokyo and Kyoto. Since the number of bags is less than 10, sorting is performed sequentially.

Java (Advanced)
Coding
100 score
40 mins
15 test cases
Java, Problem Solving, Multithreading, Fork-Join

Question

Implement an airport baggage sorting system that efficiently categorizes items by destination using parallel processing.
Each baggage item, identified by its name and destination, should be grouped accordingly, leveraging the Fork-Join Framework for parallel sorting.

Goal

- Sort a list of baggage items by their assigned destinations using either sequential sorting (sorting via ascending order of their arrival time) or a divide-and-conquer approach based on the constraints outlined in the *compute()* function description.
- Display each destination, sorted alphabetically in ascending order, along with its assigned baggage names sorted as per the given defined rule.

Note:

- It is guaranteed that there are no duplicate baggage or destination names.
- Each bag's destination is included in the given list of destinations.

Example

```
3
2
Paris
Berlin
BagA
Paris
BagC
Berlin
BagB
Berlin
```

Explanation:

- *numberOfBags*: 3
- *numberOfDestinations*: 2
- Destinations: ["Paris", "Berlin"]
- Bag Names: ["BagA", "BagB", "BagC"]
- Destinations for each Bag: ["Paris", "Berlin", "Berlin"]

Output:

```
Sorted Baggage by Destination:
Berlin: [BagC: Berlin, BagB: Berlin]
Paris: [BagA: Paris]
```

Explanation:
The bags are categorized into two destinations: "Berlin" and "Paris".

- Since there are fewer than 10 bags, sorting is sequential.

The following sequence of events unfolds:

- BagA arrives 1st with its destination as "Paris".
- BagC arrives 2nd with its destination as "Berlin".
- BagB arrives 3rd with its destination as "Berlin".

Since BagC arrives before BagB, the destination "Berlin" will contain bags in the following order [BagC, BagB].
Only one baggage item has the destination "Paris"; thus, it will contain [BagA].

Classes and Methods:
class BaggageItem : Each object of this class contains:

- *name*: The bag's name, e.g., "BagA".
- *destination*: The bag's destination, e.g., "Paris".

class BaggageSorter extends RecursiveTask<Map<String, List<BaggageItem>>>:
It handles parallel sorting of baggage items using the Fork-Join Framework, extending *RecursiveTask* for recursive task decomposition.

Its attributes are:

- *baggageItems*: A list of all baggage items to sort.
- *start*: The start index of the list for the current task.
- *end*: The end index of the list for the current task.

Map<String, List<BaggageItem>> sortBaggage(int numberOfBags, List<BaggageItem> baggageItems): The method manages parallel sorting of baggage items.

- It creates a *ForkJoinPool*.
- It invokes the *BaggageSorter* task to perform parallel sorting.
- It returns a map with destination names as keys and sorted lists of corresponding baggage items as values.

Function Description
Complete the function *compute()* in the editor: It sorts the baggage items based on their destinations either using sequential sorting or in parallel using the Fork-Join Framework.

- If the task is small (10 or fewer baggage items): Sort sequentially i.e., sorting as per the baggage arrival time.
- If there are more than 10 baggage items, split the task into left and right subtasks, sort both in parallel, merge the left into the right, i.e., *right = right + left*, and return the right subtask as the final sorted result.

Function Parameters:
No parameters, but the class has access to *baggageItems*, *start*, and *end* to define the scope of work for each subtask.

Returns

Map<String, List<BaggageItem>>: where the key is a destination, e.g., "Paris", and the value is a list of baggage items assigned to that destination.

Constraints

- 1 ≤ numberOfBags ≤ 30
- 1 ≤ numberOfDestinations ≤ 20
- Each bag's name will be a non-empty string that may contain a single whitespace with 1 to 20 alphanumeric characters.

The first line contains the *numberOfBags*.
The second line contains the *numberOfDestinations*.
The next *numberOfDestinations* lines include the destination names.
The next *2*numberOfBags* lines contain baggage items. For each baggage item, two lines follow:
The first line contains the *name*.
The second line contains the *destination*.

Sample Input 0

```
STDIN       Function
-----       --------
4           numberOfBags = 4
2           numberOfDestinations = 2
Miami       destinations = ["Miami", "Dallas"]
Dallas
Bag1        bag names = ["Bag1","Bag2","Bag3","Bag4"}
Miami       bag destinations = ["Miami","Dallas","Miami","Dallas"]
Bag2
Dallas
Bag3
Miami
Bag4
Dallas
```

Sample Output 0

```
Sorted Baggage by Destination:
Dallas: [Bag2: Dallas, Bag4: Dallas]
Miami: [Bag1: Miami, Bag3: Miami]
```

Explanation

- The baggage is categorized into 2 destinations: Miami and Dallas.
- Since the number of items is less than 10, sorting is performed sequentially.

Sample Input 1

```
3
```

```
2
Tokyo
Kyoto
BagX
Tokyo
BagY
Kyoto
BagZ
Tokyo
```

Sample Output 1

```
Sorted Baggage by Destination:
Kyoto: [BagY: Kyoto]
Tokyo: [BagX: Tokyo, BagZ: Tokyo]
```

Explanation

- There are 3 baggage items: BagX, BagY, and BagZ.
- There are 2 destinations: Tokyo and Kyoto.
- Since the number of bags is less than 10, sorting is performed sequentially.